

Continue, Abort, or Fall: Viability-Aware Policy Selection (VAPS) for Safe Humanoid Acrobatics

Anonymous Authors

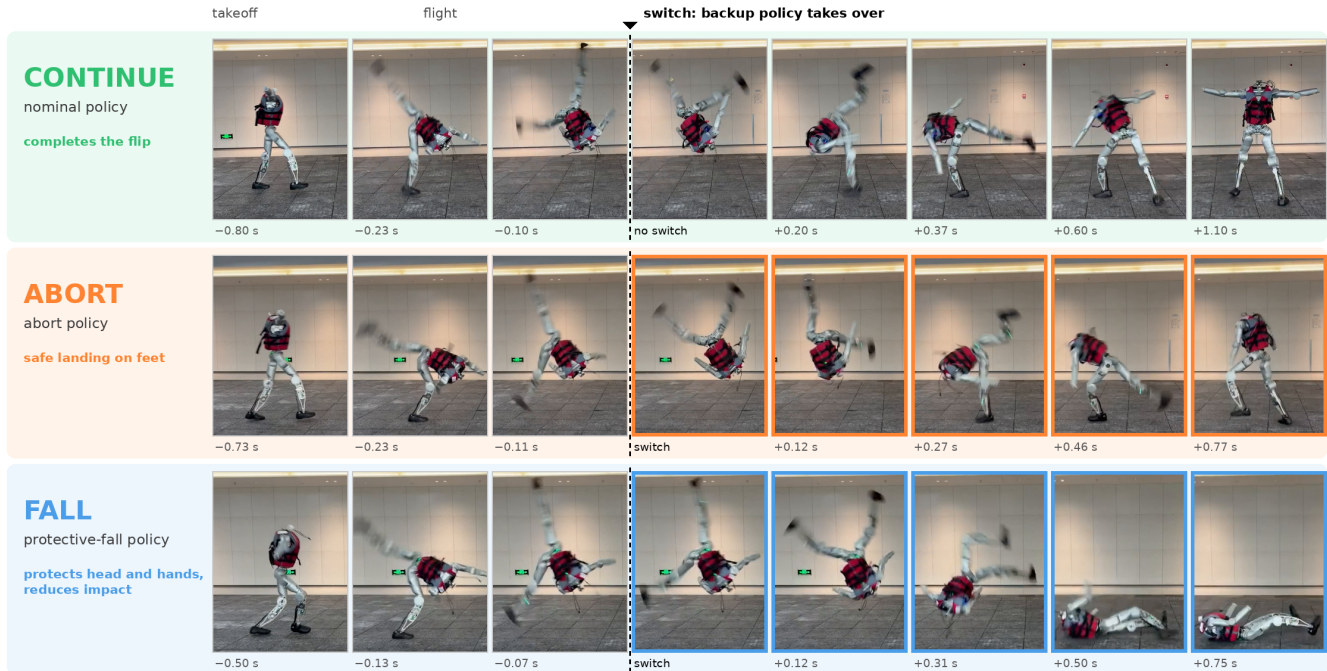


Fig. 1: The same side flip on the LimX Oli humanoid with three possible endings. Under the nominal policy the robot completes the flip; when the maneuver is handed mid-flight to the abort policy it lands on its feet, and when it is handed to the protective-fall policy it performs a controlled fall protecting head and hands.

Abstract—Dynamic humanoid motions such as flips risk hardware damage, due to suboptimal policies, disturbances or sim-to-real gaps. A motion tracking policy offers no way out once the maneuver leaves its reference, and a backup policy needs to take over to protect the hardware for a least-damage landing. Which backup to use matters as much as when to switch. We present Viability-Aware Policy Selection (VAPS), which treats safety as a policy-conditioned, receding-horizon decision. Besides a protective fall policy, we also train an abort policy which can abort the motion at any time, landing with feet. At every control step, learned predictors estimate whether the nominal tracking policy and the abort policy remain viable over a short horizon, and a least-sacrificial hierarchy keeps the most task-ambitious behavior that remains viable. In simulation with randomized disturbances, VAPS sharply reduces head contact and hand contact, which are the dominant sources of hardware damage, with both a Unitree G1 and a LimX Oli; on the LimX Oli, we validate the viability predictors and the full VAPS controller for side-flip motions. VAPS Pareto-dominates the strongest single-network alternatives we could train, including an end-to-end safe-tracking policy and students distilled from VAPS’s own oracle-routed decisions, on the task-success/head-impact frontier. We also show that VAPS is a powerful framework to supervise undertrained policies and protect the hardware.

I. INTRODUCTION

Humanoid robots are increasingly capable of highly dynamic whole-body behaviors, including running, jumping, and acrobatics [1], [2], but deploying such maneuvers safely remains difficult. Sim-to-real gaps, disturbances or suboptimal policies are common reasons for deployment failure and cause significant hardware damage, especially for highly dynamic acrobatic motions.

Humanoid acrobatics are typically learned as reference-tracking policies[2]. A tracking policy is rewarded for staying close to its reference, and its training episodes terminate as soon as it strays from it; at deployment, once the maneuver has left the reference, a backup must take over. Most existing safety mechanisms nonetheless cast this intervention as a binary choice between continuing the nominal controller and activating a single backup policy or safety shield, which either steers the robot back to a safe region[3], [4], [5] or executes a safe fall[6]. Highly dynamic acrobatics, however, are fast and short, lasting only one to two seconds. When something goes wrong, there is little time left to correct or recover, only to abort safely. Depending on the severity of the deviation, a robot that has left its reference may still

be able to abort the motion with a controlled landing on its feet rather than a fall, whereas even a learned safe fall can cause serious damage at these speeds. A single switch is therefore too coarse, particularly in the intermediate regime where the maneuver is already lost but a controlled landing is not. Folding execution and recovery into one policy [7], [8] does not restore the missing granularity either: how much of the task to sacrifice is then fixed implicitly in one set of weights, where it is neither inspectable nor, in our experiments, actually learned.

We address these challenges with *policy-conditioned, receding-horizon viability*: whether a state is safe depends on which closed-loop behavior runs next, and should be judged over a short, continuously re-evaluated window rather than by a single long-horizon guess. We instantiate this as Viability-Aware Policy Selection (VAPS), which selects among the *nominal* tracking policy and backups ordered by decreasing task ambition: an *abort* policy that seeks a controlled feet-first landing, and a *protective-fall* policy that minimizes harm once upright recovery is no longer viable. A policy-dependent predictor, which encodes a short observation history, re-estimates each behavior’s short-horizon viability at every control step, and VAPS preserves the most ambitious behavior that remains viable. Because the predictor judges the state and the candidate behavior rather than the cause of risk, the same mechanism recovers a mature policy from disturbances, supervises a new and unproven one, and serves a stop demanded from outside the robot by an operator or a workspace monitor. As hardware failures are expensive to collect, all failure cases are generated in simulation, and each state is replayed under every candidate behavior.

We summarize our main contributions as follows:

- *Least-sacrificial safety*. VAPS formulates intervention during a dynamic maneuver as a selection over an ordered hierarchy of behaviors: continue, abort, and protective fall, committing to the smallest sacrifice of task ambition that remains viable.
- *Policy-conditioned, receding-horizon viability*. We learn one viability predictor per backup from counterfactual rollouts and evaluate it continuously from proprioceptive history.
- *Analysis of single-policy alternatives*. We investigate monolithic alternatives, including end-to-end RL policies and students distilled from VAPS with DAGger, and show that VAPS dominates all of them.
- *Hardware validation*. We deploy and validate VAPS on the LimX Oli humanoid, demonstrating successful cross-robot and sim-to-real transfer.

II. RELATED WORK

A safety architecture for a dynamic maneuver must supply four things: a controller that executes the motion, behaviors that protect the robot when it fails, a signal that says when to intervene, and a structure that organizes the intervention. We review the literature on each in turn and then position VAPS against all four.

Humanoid motion tracking and acrobatics. Reference-conditioned reinforcement learning lets humanoids reproduce complex motions from human demonstrations, from phase-conditioned tracking [9] to large-scale retargeting, privileged learning, and sim-to-real transfer [10], [1], [2], and now covers jumping, spinning, cartwheels, and parkour [2], [11], [12], [13]. These methods enable the motion, but what the robot should do once the reference is infeasible is not posed.

Protective and recovery behaviors. Fall protection spans model-based contact selection that minimizes impact impulse [14], learned protective policies triggered by a continuously running fall predictor [6], curriculum-discovered protective postures [15], and robot-agnostic stylized falling [16]. These supply the behavior a robot needs once the maneuver is lost, and supply it well. What none of them poses is a choice: each acts only when falling is already unavoidable, and each offers a single response, so the band in which the maneuver is lost but a controlled landing is not has no behavior of its own.

Deciding when to intervene. The trigger has been cast as a learned shield or constraint critic [4], [3], as control barrier functions and reachability [17], [18], as a model-predictive safety filter [19], as Hamilton–Jacobi values scaled to humanoids by a geometrically distributed receding horizon [20], and, closest to humanoid falls, as a continuously running fall predictor [6]. Each computes one safety value for one policy. A quantity of that form cannot separate the two questions a lost maneuver raises – whether the task will fail, and whether recovery is still possible – and a fall/no-fall label collapses them by construction.

Organizing the intervention. Given a signal, the common structure is binary handover from the task policy to a recovery policy, whether triggered by a constraint critic [3], by predicted violations on a legged robot [5], or by a reach-avoid backup in a hierarchical controller [21]; these designs assume one nominal objective, one generic recovery behavior, and a mature nominal policy [22]. Closest to our setting, concurrent work on humanoid locomotion learns a task-agnostic stoppability value and, on a commanded stop, chooses between halting upright and a damping fall [23]. A growing line removes the structure instead, blending execution and recovery in one policy: martial-arts motions, flips included, that recover from disturbed or fallen states [7]; fall prevention, impact mitigation, and standing recovery unified [8] and distilled into a vision-based student [24]; or locomotion learned jointly with recovery [25], [26]. Pairing acrobatics with recovery is therefore not new; what these policies fix in one set of weights is how much of the task to sacrifice.

Positioning. VAPS takes the first line as given: the nominal policy is whichever tracking controller is deployed, mature or not. From the second, it trains two backup specialists rather than one: an abort policy that aims for a controlled landing, and a protective-fall policy as the last resort. Its departure from the third line is twofold. Where prior work computes one safety value for one policy, a per-policy predictor re-estimates each candidate behavior’s finite-

horizon viability at every control step; and where a switch hands one policy over to one backup, VAPS keeps several named behaviors (continue, abort, fall) and retains the most ambitious one that is still viable. We also investigate the single-policy alternatives to VAPS.

III. METHOD

VAPS has three parts, illustrated in Fig. 2. A set of specialist policies covers the maneuver and the two ways out of it, each trained on the states in which it is actually entered (§III-A). For each of the nominal and abort policies, a dedicated viability predictor estimates, from a short observation history, whether that policy can still meet its own objective over a finite horizon (§III-B). A least-sacrificial selection rule reads these estimates at every control step and retains the most task-ambitious behavior that remains viable (§III-C). In addition, §III-D investigates single-network alternatives to this decomposition.

A. Specialist Policies

Humanoid acrobatic motions are learned by imitating reference motions captured from humans and retargeted to each robot[2]. To keep the policy on its reference, every training episode is terminated as soon as the robot deviates from it. The resulting policy has never seen a departure from the reference, so it cannot save the robot once one occurs at deployment: a backup policy has to take over.

The states at which control is handed over differ in how dangerous they are, and different backup behaviors from the same state lead to different chances of damage. We therefore define two backup behaviors with conflicting objectives: an abort policy that seeks a controlled, feet-first landing, and a protective-fall policy that limits harm once a fall has become unavoidable. Together with the nominal controller, this decomposes the safe execution of an acrobatic motion into three specialist behaviors with separate objectives: nominal maneuver execution, controlled abort, and protective falling. The corresponding policies are

$$\Pi = \{\pi_{\text{nom}}, \pi_{\text{abort}}, \pi_{\text{protect}}\}, \quad (1)$$

labeled Nominal, Abort, and ProtFall in tables and figures.

Nominal Policy. π_{nom} denotes whichever motion-tracking policy is deployed or under evaluation. In our experiments, it tracks a single reference clip, obtained by motion capture and retargeting, under

$$r_{\text{nom}} = r_{\text{track}} - c_{\text{reg}}, \quad (2)$$

where r_{track} measures agreement with the reference joint configuration, body poses, and body velocities, and c_{reg} penalizes action rate, soft joint-limit violations, and self-collision. The structure of the maneuver enters only implicitly, through the reference clip. Because each training episode terminates once the robot deviates too far from the reference, π_{nom} is never trained on what follows a deviation and cannot serve as its own backup.

Abort Policy. π_{abort} minimizes damage by seeking a controlled landing: an upright base with only the feet in ground

contact. It is trained to reach and hold this configuration, without reference to the original maneuver, under

$$r_{\text{abort}} = r_{\text{alive}} + r_{\text{upright}} + r_{\text{support}} - c_{\text{reg}}, \quad (3)$$

where r_{alive} rewards every step that does not end in a hard termination, r_{upright} rewards keeping the base’s vertical axis upright, and r_{support} rewards simultaneous contact of both feet. Training resets are drawn from a precomputed dataset of *switch states*: simulator states snapshotted while rolling out the frozen nominal policy offline under domain-randomized disturbances, weighted toward harder-to-recover states in the flight and landing phases. The abort policy is thus trained on the handoff distribution of the very nominal policy it backs up.

Protective-Fall Policy. A controlled landing is not always possible. When a fall is unavoidable, π_{protect} takes over to reduce its severity. Two parts of a humanoid need particular protection: the head, which carries cameras and other easily broken sensors, and the hands, which are dexterous, fragile, and expensive. While shielding these, the robot should fall so as to minimize contact impacts and the joint loads they induce. π_{protect} is trained from the same switch states, concentrated around flight and landing, under

$$r_{\text{protect}} = r_{\text{alive}} - c_{\text{head}} - c_{\text{hand}} - c_{\text{joint}} - c_{\text{torque}}, \quad (4)$$

where c_{head} and c_{hand} penalize contact forces on the head and hand regions beyond the robot’s static support load, and c_{joint} and c_{torque} penalize reaction-force and torque overload relative to per-joint-group thresholds. Rollouts end only at timeout or on head-region contact, so success for π_{protect} means mitigated harm rather than task completion.

B. Policy-Conditioned Receding-Horizon Viability

With the specialists in place, what remains is the signal that decides between them. Let $F(s)$ be a failure predicate on states and, for each policy π_i , $i \in \{\text{nom}, \text{abort}\}$, let $\Delta_i(s_t)$ be the number of control steps until F first fires when π_i is run from s_t . The H -step viability is

$$V_i^H(s_t) = 1 - \Pr(\Delta_i(s_t) \leq H \mid s_t, \pi_i), \quad (5)$$

re-estimated at every control step.

Failure labels. The predicate must describe a physical outcome, not a training artifact. The tracking-error terminations that shape π_{nom} fire on deviations the robot often survives and cut the episode before any fall is observable, so a predictor trained against them learns the maneuver’s phase statistics rather than its physics. We therefore set F to a *physical fall* – terrain contact with any non-foot collision geometry, which is also what our evaluation scores – and read the same F under both policies. This keeps the two estimates on a common scale; what differs is only the behavior they are conditioned on.

Labels come from simulation. Each policy is rolled out under domain randomization and injected disturbances from the states at which it is actually entered – the nominal from disturbed states along the maneuver, the abort from

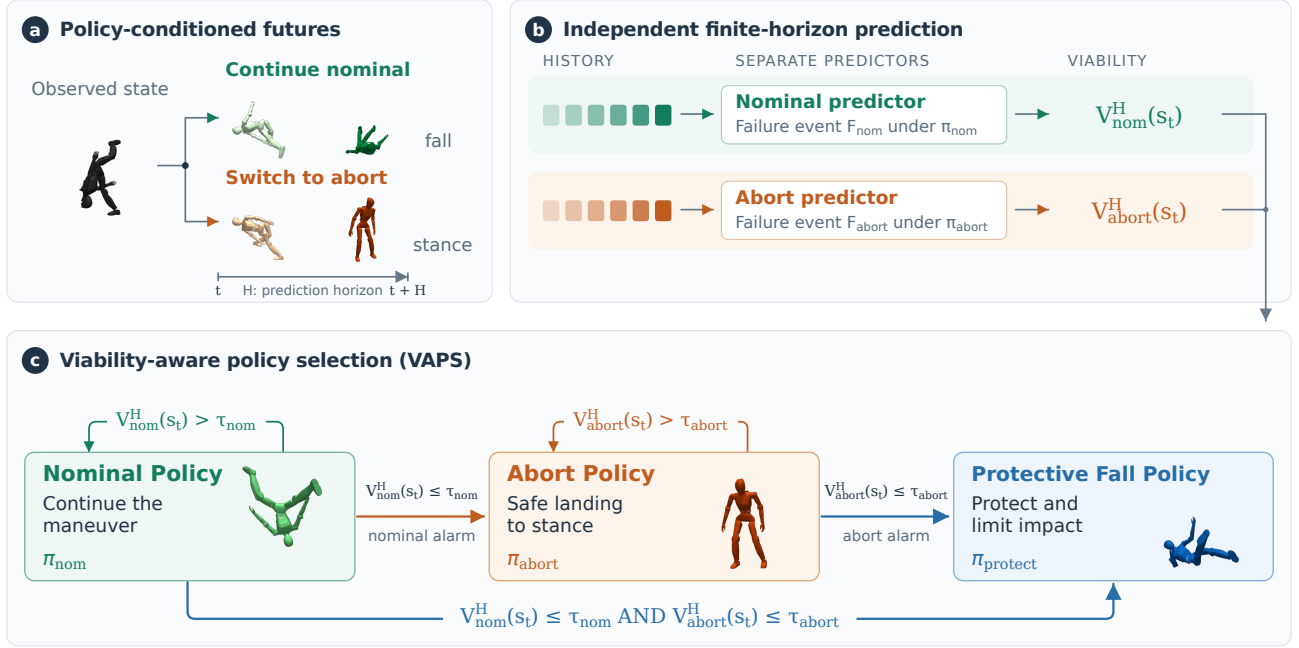


Fig. 2: Overview of VAPS. (a) Imaginary policy-conditioned futures from counterfactual training. (b) Policy-conditioned receding horizon viabilities. (c) VAPS: The least-sacrificial rule of policy selection.

the alarm’s handoff states – and each episode is ended by F , giving the targets

$$y_{t,i}^H = \mathbb{I}[\Delta_i(s_t) \leq H]. \quad (6)$$

Time-discounted supervision. Eq. 6 is a step function in time, so lengthening H to buy warning time floods the positive class with distant, weakly informative windows. We instead keep a long horizon and weight each positive window by its time to failure,

$$w(\Delta) = \lambda^{\Delta-1}/\bar{w}, \quad \lambda \in (0, 1], \quad (7)$$

with negatives at unit weight and $\bar{w}$ the mean positive weight, so that λ changes the shape of the supervision but not the class balance. The predictor then converges to a λ -discounted hazard rather than to $\Pr(\Delta_i \leq H)$, and $\lambda=1$ recovers the undiscounted target exactly. Because the output is a hazard rather than a calibrated probability, its threshold carries no probabilistic meaning and is set empirically (§III-C).

Predictors. Each candidate policy has its own predictor, which estimates the failure hazard from a trailing window of the N most recent observations rather than from the instantaneous state,

$$\hat{h}_i(s_t) = f_{\theta_i}(o_{t-N+1:t}), \quad \hat{V}_i^H(s_t) = 1 - \hat{h}_i(s_t), \quad (8)$$

where o_t is the observation available to π_i – not the full simulator state, which keeps the predictor deployable – and f_{θ_i} has a scalar sigmoid readout, trained on the targets of

Eq. 6 under the weights of Eq. 7. The history is what makes finite-horizon viability estimable at all: the instantaneous state does not separate a recoverable configuration from one already committed to failure. The two predictors share this formulation and nothing else. The nominal predictor (the *trigger*) is a single-layer GRU over $N=20$ steps (0.4 s) of the policy’s 157-D observation, with $H=50$ (1 s at 50 Hz) and $\lambda=0.90$, trained on disturbed states along the maneuver at four DR scales. The abort predictor (the *gate*) is an MLP over $N=5$ steps of 106-D proprioception with $H=25$, undiscounted, trained on the handoff states the deployed alarm produces. Matching the gate’s training states to the states on which it is read mattered more than any architectural choice: it moved the gate further than the architecture, window, and sensing sweeps combined (93 configurations, all within seed spread). The protective fall is the terminal option and needs no viability estimate.

C. Viability-Aware Policy Selection

At deployment, the robot follows a least-sacrificial decision hierarchy:

$$\pi_t = \begin{cases} \pi_{\text{nom}}, & \hat{h}_{\text{nom}}(s_t) < \tau_{\text{nom}}, \\ \pi_{\text{abort}}, & \hat{h}_{\text{nom}}(s_t) \geq \tau_{\text{nom}} \wedge \hat{h}_{\text{abort}}(s_t) < \tau_{\text{abort}}, \\ \pi_{\text{protect}}, & \text{otherwise.} \end{cases} \quad (9)$$

Transitions are irreversible within an episode, yielding the ordered sequence $\pi_{\text{nom}} \rightarrow \pi_{\text{abort}} \rightarrow \pi_{\text{protect}}$. The first comparison runs at every control step and latches when it first fires; the second is read once, at that handoff, so the route out of π_{nom} is decided in one shot. The abort hazard then keeps running as a monitor, escalating an abort that loses viability to π_{protect} at a stricter threshold $\tau_{\text{esc}}=0.95$; §IV-C reports the pipeline with and without this second transition.

Neither threshold is a probability, and the two are fixed by different criteria. A $\leq 1\%$ false-alarm budget on held-out stable validation episodes selects the nominal predictor and fixes the validation operating point of §IV-B; the deployed $\tau_{\text{nom}}=0.80$ is then set on the abort’s success from the state it is handed, because detection is saturated over the usable range and the threshold moves only the timing of the handoff. $\tau_{\text{abort}}=0.13$ is a routing threshold, fitted so that the episodes it sends to the abort recover upright at a higher rate than indiscriminate routing would achieve. Because the rule reacts to predicted viability rather than to why viability is low, the same hierarchy safeguards a mature policy against disturbance and the hardware while a new policy is being tested.

D. Monolithic Alternatives

Eq. 9 makes the sacrifice explicit; the alternative is to leave it implicit in one set of weights. §IV-D tests that choice with two single-network alternatives built on the same robot, motion and reward terms, each given the advantages the setting allows.

End-to-end safe tracking absorbs the safety objective into the task reward, $r_{\text{e2e}} = r_{\text{nom}} - \lambda (c_{\text{head}} + c_{\text{joint}} + c_{\text{torque}})$, so its safety objective is the protective specialist’s and what it lacks is only the ability to switch objectives. The tracking-error terminations of π_{nom} are removed, so an episode that loses the maneuver runs on to the fall and is charged for it, and training is warm-started from the frozen π_{nom} at a reduced learning rate, since runs from scratch either shortcut the flip without inverting or decline it outright. Because λ is the single knob trading task against damage – a sensitivity well documented in constrained RL [27], [28] – we train $\lambda \in \{0, 0.3, 1, 3, 6, 10, 20, 30\}$ at five seeds each and report the best working point (§IV-D).

Distillation from VAPS copies the deployed system’s behavior instead of its objective. The teacher is an oracle-routed pipeline: at every alarmed state, counterfactual rollouts under each specialist reveal which behavior succeeds and the teacher takes it, so the student imitates a router strictly better than the deployed one (though with the trigger’s timing). Each student is a single feedforward network on the nominal policy’s observation [29], trained by supervised learning on a stratified mix of benign, pre-alarm and post-switch segments, five seeds per arm. A faithful student sees protective falling in only 12.7% of its post-switch supervision, so two further arms lift that bound deliberately: $+protect$ draws half of its post-switch frames from protective-fall segments, and $+DAgger$ [30] adds a shard of student-visited states that the oracle routes to the protective fall – both more protective than

faithful distillation would produce, the direction that favors the safety claim under test. Both families see what π_{nom} sees, one frame at a time; the observation history the predictors encode is part of what the monolithic setting forgoes.

IV. EXPERIMENTS

The experiments address five questions:

- 1) How do the individual specialists perform, in simulation and on hardware? (§IV-A)
- 2) How accurately is finite-horizon viability predicted relative to prior fall-prediction baselines, and does that accuracy hold on the real robot? (§IV-B)
- 3) What does routing on viability buy in simulation and on hardware: how much safety is gained, and at what cost in task completion? (§IV-C)
- 4) How does VAPS compare with monolithic alternatives, obtained either by end-to-end reinforcement learning or by distilling VAPS into a single policy? (§IV-D)
- 5) Can VAPS supervise partially trained policies it was not built around? (§IV-E)

Simulation in this section uses the Unitree G1 performing a front flip in MJLab at 50 Hz; hardware experiments transfer to the LimX Oli at the same rate with a side flip motion. Unless stated otherwise, results are mean $\pm$ std over five seeds.

A. Specialist Policies

Table I rolls the three policies out at three domain-randomization (DR) scales, the two backups from identical handoff states, and exposes the three regimes the method is built on. At the training scale all three succeed without touching the ground. At the intermediate scale the nominal begins to lose the maneuver, while the protective fall avoids head contact on essentially every state it is handed and the abort returns nine in ten to a stance. At the strongest scale the separation is complete: the nominal completes one episode in seven and puts its head on the ground in five of eight, the abort still returns three in five to a stance, and protective falling is the most reliable on its own criterion. The maneuver thus becomes unrecoverable long before safe recovery does, and the backups are preferable over different bands of severity.

In practice, the abort policy carries most of the load. With a well-trained nominal policy, the states at which a switch actually occurs fall overwhelmingly in the band where a controlled landing is still viable, and the abort recovers from them at a high rate; the protective fall is reached only when that band has been left. We confirmed this on hardware by triggering the switch manually on the LimX Oli: handed the maneuver mid-motion, the abort returned the robot to a stance in 17 of 19 trials.

B. Viability Prediction

We evaluate the nominal predictor on held-out rollouts spanning all DR scales, at the frozen validation threshold of §III-C, against SafeFall-style final-outcome predictors [6], which estimate whether the episode ends in a fall rather than

TABLE I: Specialist performance across DR scales.

Policy	DR	Success	Head contact	Hand contact
Nominal	1	100.0 $\pm$ 0.0%	0.0 $\pm$ 0.0%	0.0 $\pm$ 0.0%
	2	86.2 $\pm$ 1.6%	8.1 $\pm$ 1.1%	13.6 $\pm$ 1.6%
	4	15.0 $\pm$ 2.2%	62.0 $\pm$ 2.9%	82.3 $\pm$ 2.0%
Abort	1	100.0 $\pm$ 0.0%	0.0 $\pm$ 0.0%	5.0 $\pm$ 0.0%
	2	89.0 $\pm$ 8.2%	9.0 $\pm$ 6.5%	11.0 $\pm$ 7.4%
	4	62.0 $\pm$ 13.0%	17.0 $\pm$ 8.4%	26.0 $\pm$ 6.5%
ProtFall	1	100.0 $\pm$ 0.0%	0.0 $\pm$ 0.0%	4.0 $\pm$ 2.2%
	2	98.0 $\pm$ 2.7%	2.0 $\pm$ 2.7%	9.0 $\pm$ 8.9%
	4	96.0 $\pm$ 4.2%	4.0 $\pm$ 4.2%	22.0 $\pm$ 12.5%

TABLE II: Predictor comparison against SafeFall-style final-outcome baselines. FAR: false alarm rate. LT: lead time.

Method	Episode Acc. $\uparrow$	Detection $\uparrow$	FAR $\downarrow$	LT mean $\uparrow$
SafeFall ($t_2 = 100$ ms)	95.5 $\pm$ 2.1%	85.8 $\pm$ 7.0%	0.4 $\pm$ 0.8%	0.205 $\pm$ 0.012 s
SafeFall ($t_2 = 200$ ms)	95.0 $\pm$ 1.5%	83.3 $\pm$ 5.1%	0.0 $\pm$ 0.0%	0.259 $\pm$ 0.014 s
SafeFall ($t_2 = 500$ ms)	95.0 $\pm$ 0.0%	84.2 $\pm$ 1.9%	0.4 $\pm$ 0.8%	0.316 $\pm$ 0.063 s
SafeFall ($t_2 = 1000$ ms)	93.5 $\pm$ 1.6%	79.2 $\pm$ 5.1%	0.4 $\pm$ 0.8%	0.281 $\pm$ 0.046 s
VAPS ($\lambda=0.9, H=50$)	97.5 $\pm$ 1.5%	91.7 $\pm$ 5.1%	0.0 $\pm$ 0.0%	0.416 $\pm$ 0.042 s

when recovery expires, retrained on our rollouts with the same splits and the same $\leq 1\%$ false-alarm rule. At the same episode-level false-alarm rate, VAPS detects more falls and warns about 60% earlier (Table II).

For [6], pushing the its decision boundary out toward our horizon buys lead time only up to $t_2=500$ ms,¹ after which lead time regresses together with detection, because the baseline’s label rule caps how much of the falling phase can be supervised: a final-outcome label can fire only once the outcome is settled, often past the point where an upright landing is still reachable, and asking for it earlier asks the network to call an outcome the state does not yet determine. A hazard over a receding window has no such cap.

Hyperparameter for the predictor. In the sweeps behind Table II ($\lambda \in \{0.90, 0.95, 0.97, 1.00\}$ against $H \in \{10, 25, 50\}$; GRU, Transformer and MLP encoders against $N \in \{10, 20, 50\}$; five seeds per cell) the horizon is decisive – median lead more than doubles from $H=10$ to $H=50$ at unchanged detection – the discount trades two points of detection against 0.06 s of lead, and the encoder barely matters.

Real-robot evaluation. On thirteen side-flip trials with the physical LimX Oli – eight successful executions and five failures – the deployed predictor classifies every trial correctly with no false alarm, firing on each failure before the corresponding simulated termination with a mean lead of 0.544 s, consistent with simulation (Fig. 3). Thirteen trials bound the false-alarm rate only loosely, but they establish that the quantity the selector depends on survives the sim-to-real transfer.

C. Full Pipeline Evaluation

The upper block of Table III compares VAPS with two single-backup ablations obtained by restricting Eq. 9

¹ t_2 is the baseline’s labelling window: a state is labelled as falling if it lies within t_2 of the end of a failed episode. Larger t_2 asks the baseline to fire earlier.

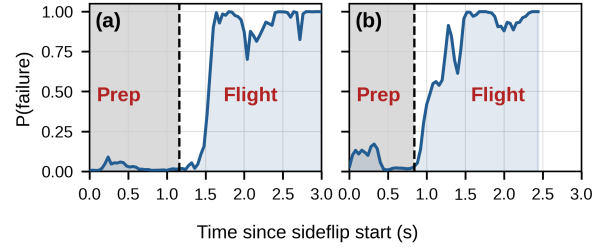


Fig. 3: Predictor output for two representative real-robot side flips. The dashed line marks take-off.

to one backup: Nominal–Abort routes every alarm to the abort, Nominal–ProtFall to the protective fall. All four rows share the same alarm rule (motion success predictor below $\tau_{\text{nom}}=0.80$), so they leave the nominal on the same episodes and motion success is identical, 59.6% against 62.1% for the unsupervised nominal. Supervision costs 2.5 points; the remaining columns show what each backup does with the alarmed episodes.

The two ablations separate the roles of the backups: the abort protects the whole body, while the protective fall keeps the head off the ground and shapes the impact. Nominal–Abort has the lowest fall rate in the table, but an abort that fails arrives in whatever attitude it lost control in, so its falls are the most damaging (median peak head force 834 N vs. 540 N unprotected) and its head contact (5.0%) the highest of any switching row. Nominal–ProtFall protects the head (2.0% head contact, median peak head force zero) but is committed to the ground by construction, so it falls more often than the nominal it supervises (40.4% vs. 37.9%).

Routing between the two keeps both. With a single switch, VAPS sends each alarm to the abort if the abort success predictor judges the state recoverable and to the protective fall otherwise: head contact stays at 2.2%, within 0.2 points of always protecting, while the fall rate drops by 14.2 points to 26.2%, consistently across all five seeds. Keeping the abort success predictor running during the abort, and escalating to the protective fall when its prediction drops, removes the residual head penalty: head contact falls to 1.6%, below the protective-fall ablation, and the mean peak head force among fallen episodes more than halves, with fall rate and motion success unchanged. The full hierarchy also records the lowest median peak non-foot force in the table.

At the handoff step, where the policy and both predictors evaluate, the three networks take on average 0.29 ms (p99 0.31 ms) on one CPU core, 1.6% of the control period.

D. VAPS versus Monolithic Policies

In this section, we turn to the second part of Table III and to Fig. 4 to ask whether the structure of VAPS, with its specialist policies and viability predictors, can be replaced by a single policy: an end-to-end RL policy, or one of several students distilled from VAPS.

Fig. 4 places every row of the table on the plane of motion success against head-contact rate, one marker per seed, and the three families occupy three distinct regions of it. The

TABLE III: Closed-loop performance under randomized disturbances. $\pm$ is over five seeds.

Method	All episodes (%)					Fallen episodes only, median [mean]	
	Success $\uparrow$	Head $\downarrow$	Hand $\downarrow$	Other $\downarrow$	Fall $\downarrow$	Pk. head (N)	Pk. non-foot (N)
Nominal only	62.1 $\pm$ 0.2	27.7 $\pm$ 0.2	36.6 $\pm$ 0.2	36.3 $\pm$ 0.1	37.9 $\pm$ 0.2	540.2 $\pm$ 13.8 [954.8 $\pm$ 122.0]	2124.6 $\pm$ 33.9 [2608.6 $\pm$ 352.2]
<i>VAPS — shared alarm rule, $\tau_{\text{nom}} = 0.80$</i>							
VAPS (Nominal–Abort)	59.6 $\pm$ 0.4	5.0 $\pm$ 0.4	7.4 $\pm$ 0.5	5.5 $\pm$ 0.3	7.4 $\pm$ 0.5	833.5 $\pm$ 139.1 [1076.0 $\pm$ 72.1]	2219.5 $\pm$ 87.2 [2110.4 $\pm$ 189.7]
VAPS (Nominal–ProtFall)	59.6 $\pm$ 0.4	2.0 $\pm$ 0.1	11.8 $\pm$ 0.6	39.9 $\pm$ 0.5	40.4 $\pm$ 0.4	0.0 $\pm$ 0.0 [21.8 $\pm$ 2.6]	1871.8 $\pm$ 14.2 [2058.6 $\pm$ 17.4]
VAPS (single switch)	59.6 $\pm$ 0.4	2.2 $\pm$ 0.2	9.3 $\pm$ 0.4	25.6 $\pm$ 1.7	26.2 $\pm$ 1.7	0.0 $\pm$ 0.0 [62.2 $\pm$ 12.9]	1855.3 $\pm$ 38.1 [2073.4 $\pm$ 77.2]
VAPS (full hierarchy)	59.6 $\pm$ 0.4	1.6 $\pm$ 0.2	8.8 $\pm$ 0.5	25.6 $\pm$ 1.8	26.1 $\pm$ 1.7	0.0 $\pm$ 0.0 [28.8 $\pm$ 3.8]	1835.7 $\pm$ 38.4 [2032.3 $\pm$ 20.9]
<i>Monolithic, memoryless — one network, no switching, one frame in</i>							
End-to-end RL ($\lambda=10$)	61.9 $\pm$ 0.9	24.0 $\pm$ 1.6	36.1 $\pm$ 1.3	36.7 $\pm$ 0.8	37.9 $\pm$ 1.0	388.2 $\pm$ 95.0 [846.7 $\pm$ 82.4]	2051.5 $\pm$ 34.5 [2372.4 $\pm$ 69.5]
Distilled (base)	57.4 $\pm$ 0.8	23.1 $\pm$ 1.1	33.6 $\pm$ 1.1	30.4 $\pm$ 0.8	34.0 $\pm$ 1.1	756.3 $\pm$ 50.7 [1100.8 $\pm$ 59.6]	2453.8 $\pm$ 33.6 [2608.8 $\pm$ 77.4]
Distilled (+protect)	58.0 $\pm$ 0.6	17.9 $\pm$ 1.4	36.7 $\pm$ 0.5	36.1 $\pm$ 0.5	38.4 $\pm$ 0.6	18.2 $\pm$ 40.8 [712.8 $\pm$ 65.2]	2329.8 $\pm$ 22.9 [2552.1 $\pm$ 22.9]
Distilled (+protect, DAGger)	56.3 $\pm$ 0.7	10.1 $\pm$ 1.3	39.7 $\pm$ 1.0	40.5 $\pm$ 0.7	42.3 $\pm$ 1.0	0.0 $\pm$ 0.0 [326.6 $\pm$ 69.3]	2132.8 $\pm$ 57.2 [2456.4 $\pm$ 54.8]
<i>Monolithic with memory — one network, no switching, history in the input or a carried state</i>							
Distilled frame-stack 5 (base)	55.5 $\pm$ 3.7	25.4 $\pm$ 0.3	39.2 $\pm$ 3.3	35.0 $\pm$ 1.2	39.7 $\pm$ 3.3	675.0 $\pm$ 129.2 [1039.2 $\pm$ 85.0]	2383.7 $\pm$ 137.6 [2498.0 $\pm$ 192.7]
Distilled GRU (base)	58.7 $\pm$ 1.1	24.2 $\pm$ 0.8	34.8 $\pm$ 1.1	32.2 $\pm$ 0.8	35.2 $\pm$ 1.1	774.2 $\pm$ 49.9 [1121.0 $\pm$ 41.9]	2482.6 $\pm$ 42.5 [2619.2 $\pm$ 52.5]
Distilled GRU (+protect)	56.0 $\pm$ 1.0	15.4 $\pm$ 0.9	35.3 $\pm$ 1.4	34.3 $\pm$ 1.9	37.0 $\pm$ 1.3	0.0 $\pm$ 0.0 [646.8 $\pm$ 55.6]	2256.1 $\pm$ 44.9 [2502.4 $\pm$ 33.8]
Distilled GRU (+protect, DAGger)	56.1 $\pm$ 1.3	17.9 $\pm$ 1.2	33.6 $\pm$ 1.4	32.2 $\pm$ 1.3	34.8 $\pm$ 1.5	227.2 $\pm$ 204.1 [835.9 $\pm$ 101.8]	2369.2 $\pm$ 61.4 [2574.4 $\pm$ 92.8]

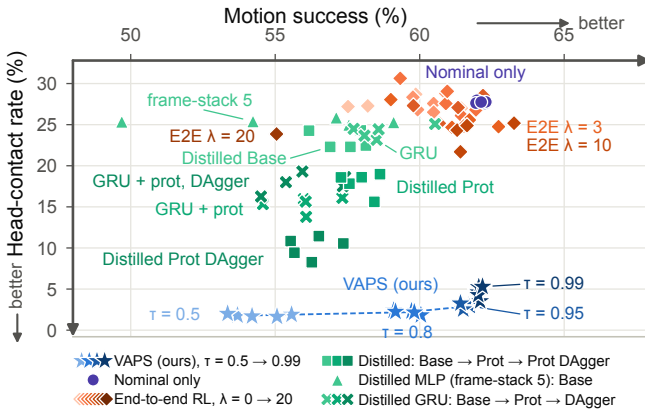


Fig. 4: Head-contact rate against motion success on the population of Table III; down and right is better.

end-to-end policies never leave the nominal’s corner: across the whole penalty ladder they lower head contact by at most four points, from 27.7 % to 24.0 %, before the weight breaks training and success scatters. The distilled students descend almost vertically, giving up little success but removing up to three-quarters of the base student’s head contact, and then plateau at about 10 %. VAPS runs along the bottom edge, where its threshold traces a frontier from about 55 % success at 2 % head contact to 62 % at 4.6 %.

The students’ plateau is the informative result. Architecture does not move it: a five-frame stack is no better than a single frame, and a recurrent state helps only once the distillation diet is protective, and then by a couple of points. What does move it is data at the switch, and asymmetrically so. The DAGger shard, whose segments begin at the alarm, takes the memoryless student from 17.9 % to 10.1 % head contact yet leaves the recurrent student where it was, because those segments begin with no history for a hidden state to carry. The students, in other words, have learned how to protect

the head; what they have not learned is when protecting it is the right thing to do. Table III shows the same thing from the other side through the outcome the figure cannot plot, an episode ending upright without completing the maneuver: VAPS produces it in fourteen points of episodes, because it commits to the abort, whereas the distilled rows stay in the low single digits, because a student that is unsure hedges the maneuver rather than choosing a backup.

The frontier makes the comparison concrete. At $\tau=0.99$, VAPS matches the nominal’s success (62.1 % against 62.2 %) at 4.6 % head contact, against 24.0 % for the best end-to-end arm and 10.1 % for the best student; every lower τ trades success for still less head contact, down to 1.6 % for the full hierarchy at $\tau=0.8$. Up to seed spread on the success axis, the non-dominated set of Fig. 4 consists of VAPS points only: the nominal and the best end-to-end seeds exceed VAPS’s success by a few tenths of a seed, at five to six times its head-contact rate. Folding the specialists into one network preserves the skills but loses the decision of how much of the task to give up, which is exactly the part the explicit structure keeps inspectable.

E. Supervising Partially Trained Policies

Because VAPS is trained on a large dataset collected from randomized and perturbed simulations, the same pipeline generalizes across policies and serve as a protective framework during the early deployment of a new policy or a new robot. To test whether it can supervise suboptimal, undertrained policies, we train the nominal policy again from scratch, save a checkpoint every 2,000 iterations, and deploy VAPS on each of the 16 checkpoints, from untrained to converged. We reuse the abort policy and the protective fall policy and only retrain the nominal predictor. Fig. 5 summarizes the outcome. In the lower panel, head contact under VAPS stays flat near the floor while that of the unprotected policy ranges from 30 to 99%; in the upper panel, the two completion curves lie almost on top of each other. VAPS thus keeps protecting the robot at every stage of training while sacrificing very little task success.

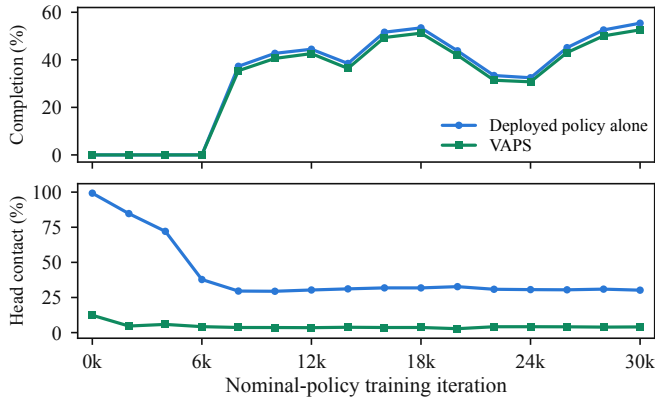


Fig. 5: VAPS around 16 checkpoints of an independently trained nominal policy.

V. CONCLUSION

We presented VAPS, which predicts safety in humanoid dynamic motions as a policy-conditioned, receding-horizon viability of several candidates behaviors.

REFERENCES

- [1] T. He, J. Gao, W. Xiao, Y. Zhang, Z. Wang, J. Wang, Z. Luo, G. He, N. Sobanbab, C. Pan, Z. Yi, G. Qu, K. Kitani, J. Hodgins, L. J. Fan, Y. Zhu, C. Liu, and G. Shi, "ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills," 2025, arXiv:2502.01143.
- [2] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, "BeyondMimic: From Motion Tracking to Versatile Humanoid Control via Guided Diffusion," 2025, arXiv:2508.08241.
- [3] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones," 2021, arXiv:2010.15920.
- [4] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," 2017, arXiv:1708.08611.
- [5] T.-Y. Yang, T. Zhang, L. Luu, S. Ha, J. Tan, and W. Yu, "Safe Reinforcement Learning for Legged Locomotion," 2022, arXiv:2203.02638.
- [6] Z. Meng, T. Liu, L. Ma, Y. Wu, R. Song, W. Zhang, and S. Huang, "SafeFall: Learning Protective Control for Humanoid Robots," 2025, arXiv:2511.18509.
- [7] Z. Lei, L. Cao, X. Wang, T. Qian, J. Liu, and X. Li, "A Kung Fu Athlete Bot That Can Do It All Day: Highly Dynamic, Balance-Challenging Motion Dataset and Autonomous Fall-Resilient Tracking," 2026, arXiv:2602.13656.
- [8] Z. Xu, Y. Li, K.-y. Lin, and S. X. Yu, "Unified Humanoid Fall-Safety Policy from a Few Demonstrations," 2025, arXiv:2511.07407.
- [9] X. B. Peng, P. Abbeel, S. Levine, and M. v. d. Panne, "DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills," 2018, arXiv:1804.02717.
- [10] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi, "OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning," 2024, arXiv:2406.08858.
- [11] Z. Zhuang, S. Yao, and H. Zhao, "Humanoid Parkour Learning," 2024, arXiv:2406.10759.
- [12] Z. Wu, X. Huang, L. Yang, Y. Zhang, X. Chen, P. Abbeel, R. Duan, A. Kanazawa, C. Sferrazza, G. Shi, and C. K. Liu, "Perceptive Humanoid Parkour: Chaining Dynamic Human Skills via Motion Matching," 2026, arXiv:2602.15827.
- [13] W. Xie, J. Han, J. Zheng, H. Li, X. Liu, J. Shi, W. Zhang, C. Bai, and X. Li, "KungfuBot: Physics-Based Humanoid Whole-Body Control for Learning Highly-Dynamic Skills," 2026, arXiv:2506.12851.
- [14] V. C. Kumar, S. Ha, and C. K. Liu, "Learning a Unified Control Policy for Safe Falling," 2017, arXiv:1703.02905.
- [15] D. Shi, S. Lyu, and D. Wang, "Discovering Self-Protective Falling Policy for Humanoid Robot via Deep Reinforcement Learning," 2025, arXiv:2512.01336.
- [16] P. Strauch, D. Müller, S. Christen, A. Serifi, R. Grandia, E. Knoop, and M. Bächer, "Robot Crash Course: Learning Soft and Stylized Falling," 2025, arXiv:2511.10635.
- [17] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [18] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi Reachability: A Brief Overview and Recent Advances," 2017.
- [19] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," 2021, arXiv:1812.05506.
- [20] R. Chen, S. Li, Y. Sun, and C. Liu, " λ -Reachability: Geometric-Horizon Safety Bellman Equations for Humanoid Safety," 2026, arXiv:2606.16022.
- [21] T. He, C. Zhang, W. Xiao, G. He, C. Liu, and G. Shi, "Agile But Safe: Learning Collision-Free High-Speed Legged Locomotion," 2024, arXiv:2401.17583.
- [22] P. Liu, K. Zhang, D. Tateo, S. Jauhri, Z. Hu, J. Peters, and G. Chalkatzaki, "Safe Reinforcement Learning of Dynamic High-Dimensional Robotic Tasks: Navigation, Manipulation, Interaction," 2023.
- [23] J. Long, P. Abbeel, K. Sreenath, R. Horowitz, G. Shi, and C. K. Liu, "Humanoid Safe Stop via Learned Stoppability Value," 2026, arXiv:2609.02358.
- [24] O. Azulay, Z. Xu, A. Scheffer, and S. X. Yu, "VIGOR: Visual Goal-In-Context Inference for Unified Humanoid Fall Safety," 2026, arXiv:2602.16511.
- [25] X. Ren, Y. Yang, Z. Weng, Z. Liu, and H. Kong, "Stubborn: A Streamlined and Unified Reinforcement Learning Framework for Robust Motion Tracking and Fall Recovery for Humanoids," 2026, arXiv:2606.12814.
- [26] Y. Lu, Y. Zhong, L. Zhao, W. Li, and P. Lu, "Unified Walking, Running, and Recovery for Humanoids via State-Dependent Adversarial Motion Priors," 2026, arXiv:2605.18611.
- [27] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained Policy Optimization," 2017, arXiv:1705.10528.
- [28] A. Stooke, J. Achiam, and P. Abbeel, "Responsive Safety in Reinforcement Learning by PID Lagrangian Methods," 2020, arXiv:2007.03964.
- [29] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [30] S. Ross, G. J. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," 2011, arXiv:1011.0686.